

Project Sentinel

Autonomous DevSecOps Swarm

From vulnerability alert to validated, human-approved fix — automatically.

Built on: **Azure AI Foundry** · **Semantic Kernel** · **AutoGen** · **Azure OpenAI**
· **GitHub Actions**

[LIVE DASHBOARD: AGENT-SWARMS-2.VERCEL.APP](https://agent-swarms-2.vercel.app)

Made with **GAMMA**

Finding vulnerabilities is solved. Fixing them safely is not.

Alert Overload

SAST and DAST tools generate thousands of alerts per week. Security teams triage — they don't fix.

Remediation Bottleneck

Every alert requires a human engineer to analyse code, write a fix, test it, and submit a PR. That loop takes days, not minutes.

Business Impact

Blocked CI/CD pipelines. Delayed releases. Alert fatigue where critical issues get ignored alongside low-priority noise.



Alert Raised

Automated scanner flags issue



Engineer Assigned

Manual analysis (2–4 days)



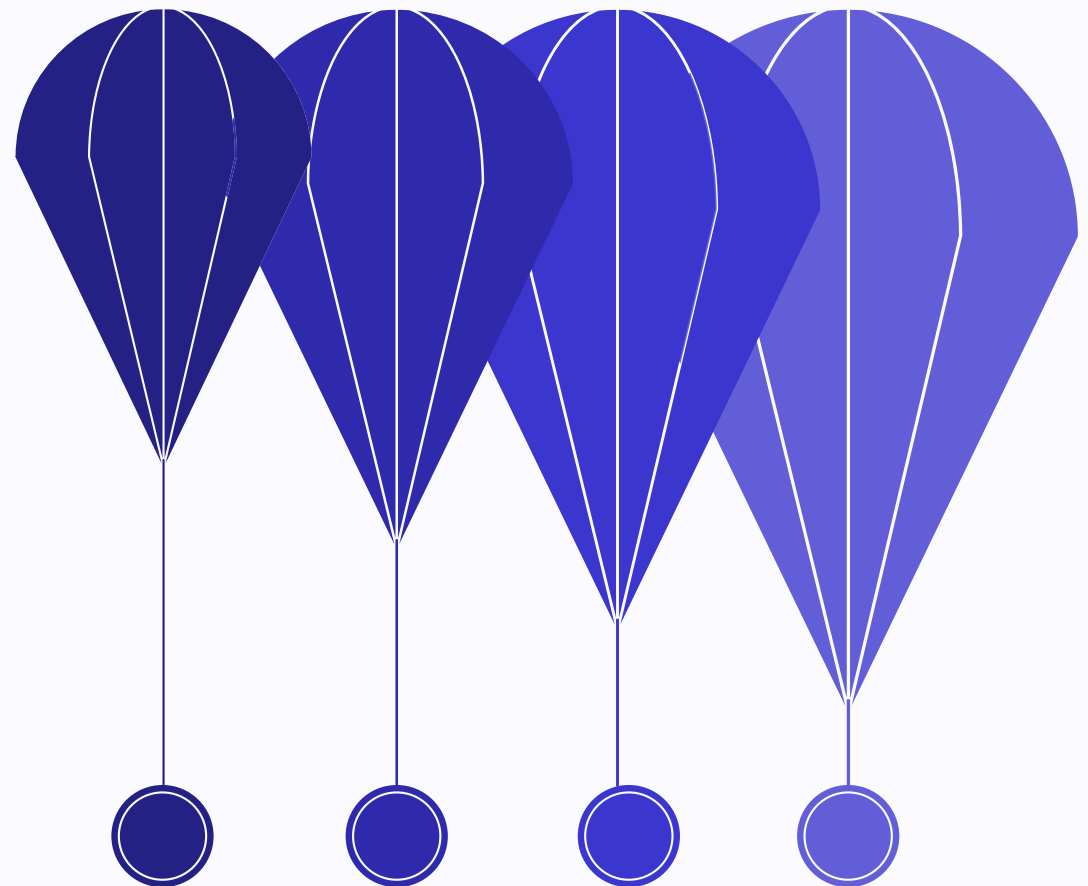
Fix Reviewed

Peer review and testing (1–3 days)



Merged to Production

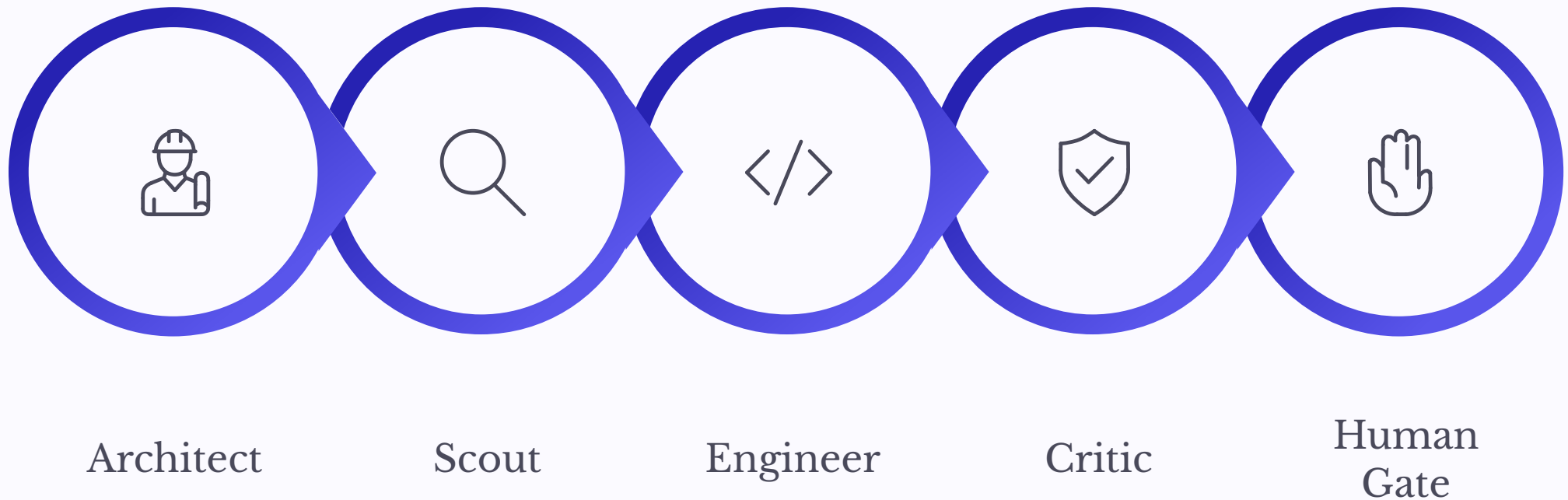
Deployment completes (1–2 days)



The gap between alert and merge is measured in days. Sentinel compresses it to minutes.

Sentinel closes the loop.

Sentinel ingests your repository, generates a context-aware patch using Azure OpenAI, validates it inside a process-isolated sandbox, and surfaces one decision to a human engineer: **Approve or Rollback.**



Nothing reaches a human until the sandbox approves it. Engineers go from writing patches to reviewing validated diffs.

Built natively on the Microsoft AI stack.

What We Use

- **Azure OpenAI (gpt-4o)** — Patch generation and Critic adversarial analysis
- **Azure AI Foundry** — Semantic code search via text-embedding-3-small
- **Semantic Kernel** — 4 agent functions registered as @kernel_function plugins
- **AutoGen GroupChat** — 4-agent audit conversation transcript per session
- **GitHub Actions** — One-file CI integration via action.yml
- **SARIF 2.1** — Findings export to GitHub Advanced Security
- **Azure DevOps** — Findings exported as work items via REST API

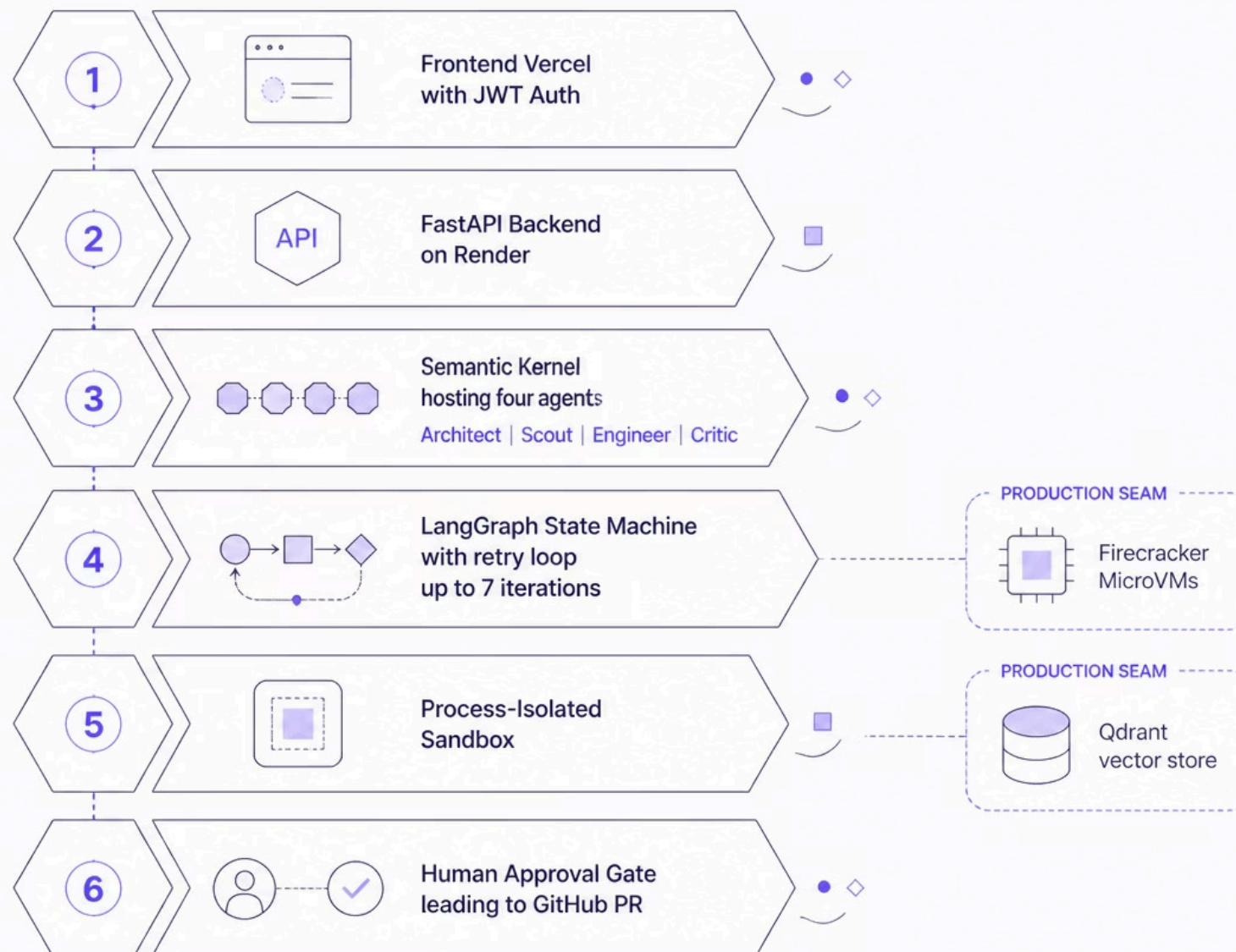
Live Endpoints — Judges Can Call These Now

- `GET /api/system/sk_status` → Semantic Kernel kernel state
- `GET /api/sessions/{id}/autogen_transcript` → AutoGen conversation log
- `GET /api/sessions/{id}/export/sarif` → SARIF 2.1 output
- `POST /api/sessions/{id}/export/azure` → Azure DevOps work items

 **Why not GitHub Copilot Autofix?** Copilot generates a patch and stops. Sentinel sandboxes it, adversarially challenges it with AutoGen, and gives you a rollback-safe approval gate.

Four agents. One convergence loop. Zero unvalidated patches in production.

The LangGraph state machine drives deterministic retry logic — if the Critic rejects a patch, the Engineer refactors and tries again, up to **7 iterations**. Azure AI Foundry embeddings give every agent semantic context, not just keyword matching.



Deployed on **Vercel** (frontend) and **Render** (backend). Production sandbox uses Firecracker MicroVMs — current deployment uses a process-isolated sandbox with the same interface. The production seam is explicitly documented at </api/system/capabilities>.

The agents don't just act — they reason, debate, and sign off.

Every audit session produces an AutoGen GroupChat transcript — four expert perspectives a human reviewer can read in 30 seconds.

Architect

"Repository analysis complete. SQL injection in users.py line 42 via f-string query construction. Data flow confirmed from untrusted HTTP parameter to database cursor."

Scout

"CWE-89 confirmed. Deterministic rule python.sql_injection.fstring matched. No encoding bypass possible — parameterisation required."

Engineer

"Patch applied: replaced f-string query with parameterised cursor.execute(). Confidence 0.91. Minimal footprint — one line changed."

Critic

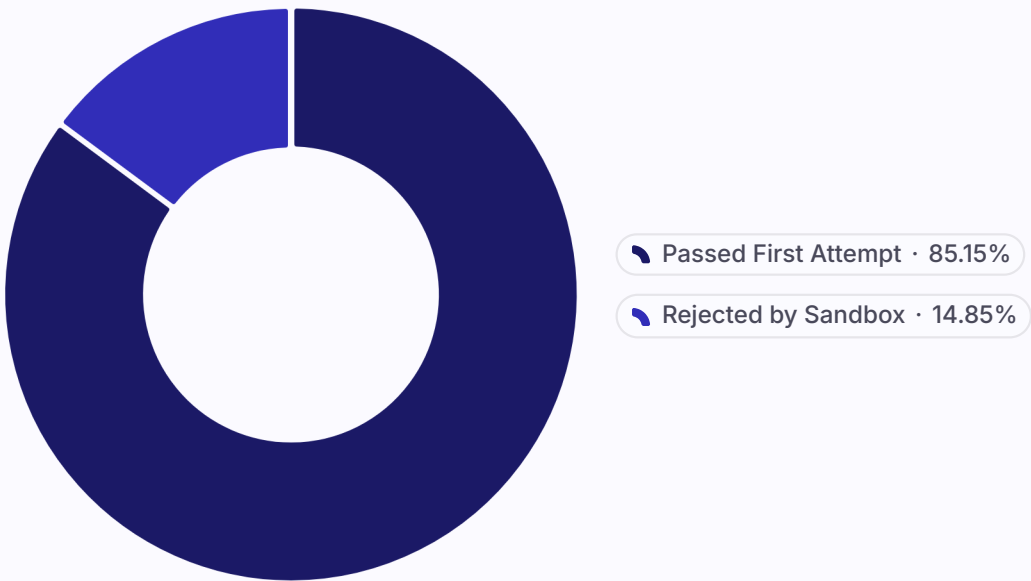
"Sandbox result: APPROVE. Tests pass. Encoding bypass and second-order injection vectors tested — both blocked. Human review recommended before merge."

Tested against 5 real vulnerable repositories.

223 findings. 150 patches generated. 86% sandbox pass rate.

Repository	Findings	Patches	Pass Rate	False Positives
examples/python-vulnerable-api	14	14	100%	0
PyCQA/bandit test fixtures	48	42	87%	3
trufflesecurity/trufflehog	32	31	96%	1
juice-shop/juice-shop	105	45	68%	12
appsec-co/dvna	24	18	80%	2
Overall	223	150	~86%	18

18 of 223 findings were escalated for human review — by design. The sandbox rejected 15% of LLM-generated patches before they could reach production.



Autonomous patching with hard guardrails.

No patch merges without passing three policy checks. If any condition fails, the patch is escalated — never silently dropped or auto-merged.

Policy Gate Logic

✓ Sandbox Verdict = APPROVE

Patch compiled and all tests passed in the isolated sandbox.

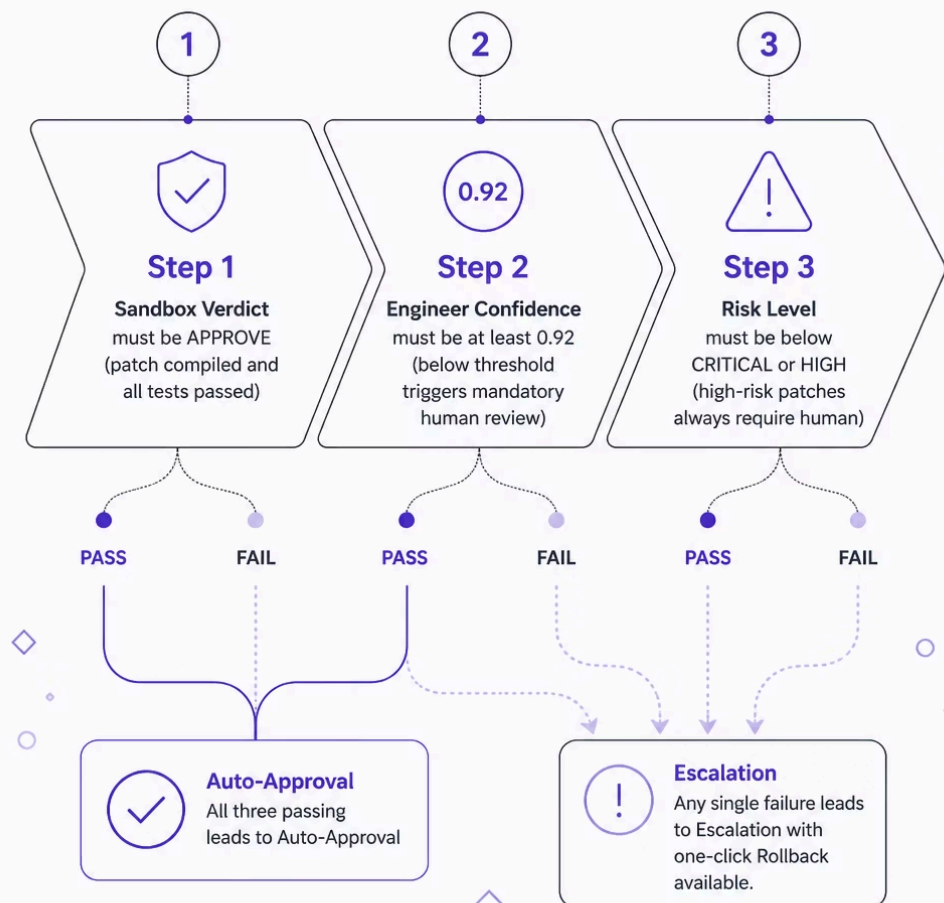
✓ Engineer Confidence ≥ 0.92

Below threshold → mandatory human review.

✓ Risk Level below CRITICAL/HIGH

High-risk patches always require a human.

❗ Rollback is always available — Sentinel stores original file content and verifies it before restoring, so engineers can revert with one click even after approval.



11 deterministic detection rules. CWE-mapped. OWASP Top 10 tagged. NIST SP 800-53 aligned.

Rule	Language	CWE	OWASP	Severity
SQL injection via f-string	Python	CWE-89	A03:2021	HIGH
Code injection via eval()/exec()	Python	CWE-94	A03:2021	CRITICAL
Path traversal	Python	CWE-22	A01:2021	HIGH
Insecure deserialization (pickle)	Python	CWE-502	A08:2021	CRITICAL
Unsafe YAML load	Python	CWE-502	A08:2021	HIGH
Weak PRNG for security	Python	CWE-338	A02:2021	MEDIUM
SQL injection via template literal	JavaScript	CWE-89	A03:2021	HIGH
XSS via innerHTML	JavaScript	CWE-79	A03:2021	HIGH
Code injection via eval()	JavaScript	CWE-94	A03:2021	CRITICAL
Hardcoded credentials	Multi	CWE-798	A07:2021	CRITICAL
Git merge conflict markers	Multi	CWE-116	A05:2021	LOW

Plus: optional **Semgrep**, **Trivy**, **Checkov**, and **Gitleaks** integration. SARIF export flows directly into GitHub Advanced Security — no integration work required.



The foundation is in place. Three phases to production scale.

1

Phase 1 — Fleet Scanning

Extend the session model to scan multiple repositories in one operation. Audit 50+ microservices simultaneously.

2

Phase 2 — Expanded Rule Coverage

Broader deterministic rules covering complex injection chains, auth bypass patterns, and infrastructure-as-code misconfigurations via Checkov integration.

3

Phase 3 — Natural Language Policy Engine

Let security teams define policies in plain English: *"Always escalate CRITICAL findings in payment services."* No YAML. No DSL. Powered by Azure OpenAI.



Sentinel turns security alerts into reviewed, validated, rollback-safe fixes — with full human control at every gate.

[Live demo](#)

[GitHub](#)

[API health](#)

Made with **GAMMA**